

Vitest & Tests d'intégration

La qualité logicielle par l'automatisation des tests

Vitest	Supertest	Tests	TDD
--------	-----------	-------	-----

Fixtures	CI/CD	Coverage
----------	-------	----------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre la pyramide de tests et quand utiliser chaque type
- ✓ Configurer Vitest dans un projet Node.js/TypeScript
- ✓ Écrire des tests d'intégration avec Supertest sur une API Express réelle
- ✓ Concevoir des fixtures de données de test propres et réutilisables
- ✓ Tester l'isolation multi-tenant et les règles de sécurité
- ✓ Mesurer et améliorer la couverture de tests (coverage)

Table des matières

Chapitre 1 — La philosophie du test automatisé

- 1.1 Pourquoi tester ?
- 1.2 La pyramide de tests
- 1.3 Test-Driven Development
- 1.4 Le coût du bug selon sa détection

Chapitre 2 — Vitest — configuration et premiers tests

- 2.1 Vitest vs Jest
- 2.2 Configuration vitest.config.ts
- 2.3 Describe, it, expect
- 2.4 Matchers et assertions

Chapitre 3 — Supertest — tester une API HTTP

- 3.1 makeApp() sans listen()
- 3.2 Requêtes HTTP simulées
- 3.3 Authentification dans les tests
- 3.4 Assertions sur les réponses

Chapitre 4 — Fixtures et données de test

- 4.1 beforeEach / afterEach
- 4.2 Créer et supprimer des tenants
- 4.3 Ordre de suppression FK
- 4.4 Factories de données

Chapitre 5 — Tests de sécurité et coverage

- 5.1 Tests d'isolation cross-tenant
- 5.2 Tests d'authentification
- 5.3 Coverage avec V8

5.4 Intégration CI/CD

La philosophie du test automatisé

1.1 Pourquoi tester ?

Les tests automatisés sont l'assurance qualité de votre code. Sans eux, chaque modification peut introduire une régression — un bug dans une fonctionnalité qui marchait avant. Plus le projet grossit, plus cette probabilité augmente. Les tests permettent de modifier le code en confiance : si les tests passent, les fonctionnalités critiques fonctionnent toujours.

Dans un SaaS multi-tenant, les tests ont une dimension de sécurité supplémentaire. Un bug d'isolation — où les données d'un tenant sont visibles par un autre — est une violation grave qui peut mettre fin à l'entreprise. Les tests automatisés d'isolation multi-tenant détectent ces bugs avant qu'ils n'arrivent en production.

1.2 La pyramide de tests

La pyramide de tests (Mike Cohn, 2009) décrit la distribution idéale des types de tests. À la base, les tests unitaires : nombreux, rapides, testant une seule fonction ou classe en isolation. Au milieu, les tests d'intégration : testent plusieurs composants ensemble (typiquement : API + DB). Au sommet, les tests end-to-end : simulent un utilisateur réel dans un navigateur — lents et coûteux à maintenir.

La pyramide de tests — types et caractéristiques

Type	Teste quoi ?	Vitesse	Coût	Quantité
Unitaire	Une fonction isolée	Millisecondes	Faible	Nombreux
Intégration	API + Base de données	Secondes	Moyen	Moyen
End-to-End	Flux utilisateur complet	Minutes	Élevé	Peu

Pour un backend Node.js/Express avec PostgreSQL, les tests d'intégration sont les plus importants. Ils testent le comportement réel de l'API avec une vraie base de données — les règles de validation, les erreurs de contrainte SQL, les politiques RLS, les transactions. Les tests unitaires purs sont moins critiques pour une API dont la logique est principalement dans le SQL.

1.3 Le coût du bug selon sa détection

Le coût de correction d'un bug est exponentiel selon le moment où il est détecté. Un bug détecté pendant le développement (par le compilateur TypeScript ou les tests) coûte quelques minutes. Le même bug détecté en code review coûte quelques heures (lecture, discussion, correction, re-review).

En production, le coût peut être considérable : données corrompues, clients insatisfaits, perte de revenus, atteinte à la réputation.

Vitest — configuration et premiers tests

2.1 Vitest vs Jest

Jest est le framework de test le plus utilisé dans l'écosystème JavaScript. Vitest est sa version moderne, construite pour Vite. Les deux partagent la même API (describe, it, expect, mock) ce qui rend la migration triviale. Vitest est nativement compatible ESM et TypeScript sans configuration supplémentaire, et est significativement plus rapide grâce au moteur de transformation de Vite.

```
# Installation

npm install -D vitest @vitest/coverage-v8
npm install -D supertest @types/supertest

# package.json – scripts de test

{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest",
    "test:coverage": "vitest run --coverage"
  }
}
```

2.2 Configuration vitest.config.ts

La configuration Vitest est critique pour les tests d'intégration qui touchent une vraie base de données. `fileParallelism: false` est indispensable : exécuter les fichiers de test en parallèle sur la même DB de test causerait des conflits de données entre tests. Les timeouts doivent être augmentés pour les appels réseau réels.

```
// vitest.config.ts

import { defineConfig } from 'vitest/config';
import dotenv from 'dotenv';

// Charger .env.test avant les tests
dotenv.config({ path: '.env.test' });

export default defineConfig({
  test: {
```

```
// CRITIQUE : séquentiel pour éviter les conflits sur la DB de test
fileParallelism: false,

// Timeouts élevés pour les appels réseau (Supabase, Railway)
testTimeout: 20_000,
hookTimeout: 20_000,

// Reporters
reporters: ['verbose'],

// Coverage
coverage: {
  provider: 'v8',
  include: ['src/**/*.ts'],
  exclude: ['src/test/**', 'src/**/*.d.ts'],
},
},
});
```

2.3 Describe, it, expect

La structure d'un fichier de test suit le pattern AAA : Arrange (préparer les données), Act (exécuter l'action à tester), Assert (vérifier le résultat). `describe()` groupe les tests liés. `it()` (ou `test()`) décrit un comportement spécifique. `expect()` formule une assertion sur un résultat.

```
import { describe, it, expect, beforeEach, afterEach } from 'vitest';
import request from 'supertest';
import { makeApp, makeAuthHeader } from './helpers';
import { createTestTenant, deleteTestTenant } from './fixtures';

describe('GET /products', () => {
  let tenant: TestTenant;

  beforeEach(async () => {
    tenant = await createTestTenant(); // Arrange
  });

  afterEach(async () => {
    await deleteTestTenant(tenant); // Nettoyage garanti
  });

  it('returns 401 without token', async () => {
```



```
// Act

const res = await request(makeApp()).get('/products');

// Assert

expect(res.status).toBe(401);
expect(res.body.error).toBeDefined();
});

it('returns empty list for new tenant', async () => {
  const res = await request(makeApp())
    .get('/products')
    .set(makeAuthHeader(tenant.tenantId, tenant.userId));

  expect(res.status).toBe(200);
  expect(res.body).toEqual([]);
});
});
```

Supertest — tester une API HTTP

3.1 makeApp() sans listen()

La convention `makeApp()` est le pattern recommandé pour tester une API Express. Au lieu de démarrer un vrai serveur HTTP (`app.listen()`), on instancie l'app Express sans l'attacher à un port. Supertest crée son propre serveur HTTP éphémère pour chaque test. Cela évite les conflits de port et accélère les tests.

```
// src/test/helpers.ts

import type { Express } from 'express';
import jwt from 'jsonwebtoken';
import { createApp } from '../app';

// Instancier l'app sans listen()
export function makeApp(): Express {
  return createApp();
}

// Forger un JWT valide pour les tests
export function makeAuthHeader(tenantId: string, userId: string) {
  const token = jwt.sign(
    { sub: userId, tenant_id: tenantId, aud: 'authenticated', role: 'authenticated' },
    process.env.SUPABASE_JWT_SECRET!,
    { expiresIn: '1h' }
  );
  return { Authorization: `Bearer ${token}` };
}
```

3.2 Assertions complètes

```
// Test complet d'une route PATCH

it('transitions order from confirmed to processing', async () => {
  // Arrange

  const order = await createOrder(tenant.tenantId, { status: 'confirmed' });

  // Act
```

```

const res = await request(makeApp())
  .patch(`/orders/${order.id}`)
  .set(makeAuthHeader(tenant.tenantId, tenant.userId))
  .send({ status: 'processing' });

// Assert – statut HTTP
expect(res.status).toBe(200);

// Assert – corps de la réponse
expect(res.body.id).toBe(order.id);
expect(res.body.status).toBe('processing');

// Assert – état en base de données
const { data } = await supabaseAdmin
  .from('orders')
  .select('status')
  .eq('id', order.id)
  .single();
expect(data?.status).toBe('processing');
});

// Test de transition invalide
it('rejects invalid status transition', async () => {
  const order = await createOrder(tenant.tenantId, { status: 'new' });

  const res = await request(makeApp())
    .patch(`/orders/${order.id}`)
    .set(makeAuthHeader(tenant.tenantId, tenant.userId))
    .send({ status: 'delivered' }); // new → delivered est invalide

  expect(res.status).toBe(409); // Conflict
});

```

Fixtures et données de test

4.1 Principes des fixtures

Une fixture est un ensemble de données de test dans un état connu. Le principe d'un bon jeu de fixtures est l'isolation : chaque test doit créer ses propres données et les nettoyer après. Partager des données entre tests crée des dépendances qui rendent les tests fragiles — un test qui modifie les données partagées peut casser les tests suivants.

```
// Cycle de vie des fixtures avec beforeEach/afterEach

describe('Order management', () => {

  let tenant: TestTenant;
  let product: Product;
  let order: Order;

  // Avant CHAQUE test – état propre garanti
  beforeEach(async () => {
    tenant = await createTestTenant();
    product = await createProduct(tenant.tenantId, { price: 5000 });
    order = await createOrder(tenant.tenantId);
    await createOrderItem(order.id, product.id, { quantity: 2 });
  });

  // Après CHAQUE test – nettoyage garanti même si le test échoue
  afterEach(async () => {
    await deleteTestTenant(tenant);
  });

  it('calculates total correctly', async () => { ... });
  it('transitions status correctly', async () => { ... });
});
```

4.2 Ordre de suppression FK

La suppression des fixtures doit respecter les contraintes de clés étrangères : les tables enfants doivent être supprimées AVANT les tables parentes. Si vous supprimez un tenant avant ses commandes, PostgreSQL refusera avec une erreur de violation de FK. L'ordre correct est l'inverse de l'ordre de

création.

```
// Ordre correct de suppression respectant les FK
export async function deleteTestTenant(tenant: TestTenant) {
  const { data: orders } = await supabaseAdmin
    .from('orders').select('id').eq('tenant_id', tenant.tenantId);

  if ((orders ?? []).length > 0) {
    // 1. Supprimer les items de commande (dépendent de orders)
    await supabaseAdmin.from('order_items')
      .delete().in('order_id', orders!.map(o => o.id));
  }

  // 2. Tables qui dépendent du tenant
  await supabaseAdmin.from('messages') .delete().eq('tenant_id', tenant.tenantId);
  await supabaseAdmin.from('orders') .delete().eq('tenant_id', tenant.tenantId);
  await supabaseAdmin.from('conversations').delete().eq('tenant_id', tenant.tenantId);
  await supabaseAdmin.from('products') .delete().eq('tenant_id', tenant.tenantId);

  // 3. Supprimer l'utilisateur auth (cascade sur public.users)
  await supabaseAdmin.auth.admin.deleteUser(tenant.userId);

  // 4. Enfin supprimer le tenant
  await supabaseAdmin.from('tenants').delete().eq('id', tenant.tenantId);
}
```

Tests de sécurité et coverage

5.1 Tests d'isolation cross-tenant

Les tests d'isolation sont les tests de sécurité les plus importants d'un SaaS. Ils créent deux tenants, créent des données dans l'un, et vérifient que l'autre ne peut pas y accéder. Trois vecteurs doivent être testés : lecture directe par ID (GET /orders/:id), listing (GET /orders doit filtrer), et modification (PATCH /orders/:id d'un autre tenant doit retourner 404).

5.2 Coverage

La couverture de tests (coverage) mesure le pourcentage de lignes de code exécutées par les tests. Un coverage élevé ne garantit pas la qualité des tests (on peut avoir 100% de coverage avec des tests qui n'assertent rien), mais un coverage faible révèle des zones non testées. Viser 70-80% de coverage sur les routes et services est un objectif raisonnable.

```
# Lancer les tests avec coverage

npm run test:coverage

# Résultat (extrait) :

# File | % Stmts | % Branch | % Funcs | % Lines |
# routes/orders.ts | 87.5 | 82.1 | 90.0 | 87.5 |
# routes/products.ts | 91.2 | 88.0 | 95.0 | 91.2 |
# middleware/auth.ts | 75.0 | 70.0 | 80.0 | 75.0 |

# Dans vitest.config.ts – seuil de coverage minimum

coverage: {
  thresholds: {
    lines: 70,
    functions: 80,
    branches: 65,
  }
}

# Si le seuil n'est pas atteint, le CI échoue
```

■ À retenir

- ✓ Les tests d'intégration sur vraie DB sont plus importants que les tests unitaires pour une API

- ✓ fileParallelism: false est obligatoire pour les tests qui touchent la DB de test
- ✓ Pattern AAA : Arrange (fixtures) → Act (requête) → Assert (résultat + DB)
- ✓ Chaque test crée ses données avec beforeEach et les nettoie avec afterEach
- ✓ Les tests cross-tenant sont les tests de sécurité les plus critiques d'un SaaS
- ✓ Coverage : viser 70-80% sur routes et services, pas 100% à tout prix

—■ Exercices

Exercice 1 — Suite de tests CRUD

Écrire une suite complète de tests pour une ressource CRUD.

- Test : GET /products → 200 + liste vide pour un nouveau tenant
- Test : POST /products → 201 + produit créé avec les bonnes propriétés
- Test : PATCH /products/:id → 200 + modification persistée en DB
- Test : DELETE /products/:id → soft delete (deleted_at renseigné)

Exercice 2 — Tests de sécurité

Écrire les tests de sécurité essentiels pour l'API.

- 401 sans token sur toutes les routes protégées
- 401 avec un token malformé ou expiré
- 403 pour un rôle insuffisant (ex: agent tente de supprimer)
- 404 cross-tenant : un tenant essaie d'accéder aux données d'un autre

Exercice 3 — Fixtures factories

Créer des factories de données de test réutilisables.

- createTestTenant() : tenant + merchant user
- createProduct(tenantId, overrides) : avec valeurs par défaut sensées
- createOrder(tenantId, overrides) : avec status configurable
- createOrderItem(orderId, productId, overrides) : item de commande

Bibliographie & Ressources

Documentation

- vitest.dev — documentation Vitest
- github.com/ladjs/supertest — documentation Supertest
- jestjs.io/docs/expect — référence des matchers (compatible Vitest)

Livres et articles

- Kent C. Dodds — testingjavascript.com — cours Testing JavaScript
- Martin Fowler — 'TestPyramid' — martinfowler.com/bliki/TestPyramid.html
- Vladimir Khorikov — Unit Testing: Principles, Practices, Patterns (Manning)